

Normalization from 1NF to 5NF: Summary Table and Step-by-Step Examples

Summary Table of Normal Forms

NF	What It Removes	Bad Pattern (Easy Notation)	Rule (When OK)	Fix (Decomposition Pattern)
1NF	Non-atomic or multi-valued attributes	(No FD issue; problem is value form)	All attributes must be atomic.	Split repeating groups: $R \rightarrow R_1, R_2$.
2NF	Partial dependency on a composite key	$(A + B) \rightarrow C$ but actually $A \rightarrow C$, where C is non-key.	No non-key attribute depends on a proper subset of a candidate key.	For $R(A, B, C)$: $R \rightarrow R_1(A, C), R_2(A, B)$.
3NF	Transitive dependency on a key	$A \rightarrow B, B \rightarrow C$ with C non-key.	For every FD $X \rightarrow Y$: X is a key or each attribute in Y is prime.	For $R(A, B, C)$: $R \rightarrow R_1(A, B), R_2(B, C)$.
BCNF	Any FD whose left side is not a key	$X \rightarrow Y$ but X is not a key.	Every determinant must be a superkey.	$R \rightarrow R_1(X, Y), R_2(\text{rest})$.
4NF	Independent multivalued dependencies	$A \twoheadrightarrow B$ and $A \twoheadrightarrow C$ with B independent of C .	For every nontrivial MVD $A \twoheadrightarrow B$, A must be a superkey.	$R(A, B, C) \rightarrow R_1(A, B), R_2(A, C)$.
5NF	Join dependencies not implied by keys	Nontrivial join dependency $JD^*(R_1, R_2, R_3)$ with no key in any part.	Every nontrivial join dependency must include a projection containing a key.	$R \rightarrow R_1, R_2, R_3$.

Table 1: Normal forms 1NF–5NF: problem, bad pattern, rule, and typical fix.

Worked Example: From Non-1NF to BCNF

We use a single example relation that is gradually transformed.

Step 0: Non-1NF Table (Repeating Group)

Consider a table `ENROLL_RAW`:

StudentID	StudentName	DeptName	CoursesList
1	Alice	CS	{CS101, CS102}
2	Bob	MATH	{MATH155}

Here `CoursesList` is a set of course codes in one cell, so the table is not in 1NF.

Step 1: Convert to 1NF

We expand the repeating group into one row per student-course pair and also add more attributes, so our 1NF table is:

`ENROLL_1NF(StudentID, StudentName, CourseID, CourseName, DeptID, DeptName, DeptPhone)`

StudentID	StudentName	CourseID	CourseName	DeptID	DeptName	DeptPhone
1	Alice	CS101	Databases	CS	Comp. Sci.	555-1000
1	Alice	CS102	Algorithms	CS	Comp. Sci.	555-1000
2	Bob	MATH155	Calculus I	MATH	Mathematics	555-2000

All attributes are atomic, so the relation is in 1NF. Assume the **primary key** is (StudentID, CourseID). The following functional dependencies (FDs) hold:

$\text{StudentID} \rightarrow \text{StudentName},$
 $\text{CourseID} \rightarrow \text{CourseName}, \text{DeptID},$
 $\text{DeptID} \rightarrow \text{DeptName}, \text{DeptPhone},$
 $(\text{StudentID}, \text{CourseID}) \rightarrow \text{all attributes}.$

Step 2: Remove Partial Dependencies (2NF)

In `ENROLL_1NF`, some non-key attributes depend on only part of the composite key:

$\text{StudentID} \rightarrow \text{StudentName},$
 $\text{CourseID} \rightarrow \text{CourseName}, \text{DeptID}.$

Both right-hand sides are non-key attributes, so the table is not in 2NF.

We remove partial dependencies by separating information about students and courses:

- **STUDENT**(StudentID, StudentName)
- **COURSE_TEMP**(CourseID, CourseName, DeptID, DeptName, DeptPhone)
- **ENROLL_TEMP**(StudentID, CourseID)

Sample data:

STUDENT		COURSE_TEMP				
StudentID	StudentName	CourseID	CourseName	DeptID	DeptName	DeptPhone
1	Alice	CS101	Databases	CS	Comp. Sci.	555-1000
2	Bob	CS102	Algorithms	CS	Comp. Sci.	555-1000
		MATH155	Calculus I	MATH	Mathematics	555-2000

ENROLL_TEMP	
StudentID	CourseID
1	CS101
1	CS102
2	MATH155

Now every non-key attribute in each relation depends on the whole key, so these relations are in 2NF.

Step 3: Remove Transitive Dependencies (3NF)

COURSE_TEMP still has a transitive dependency:

$$\text{CourseID} \rightarrow \text{DeptID} \rightarrow (\text{DeptName}, \text{DeptPhone}).$$

To reach 3NF, we split department information into its own table.

- **COURSE(CourseID, CourseName, DeptID)**
- **DEPARTMENT(DeptID, DeptName, DeptPhone)**

Sample data:

COURSE			DEPARTMENT		
CourseID	CourseName	DeptID	DeptID	DeptName	DeptPhone
CS101	Databases	CS	CS	Comp. Sci.	555-1000
CS102	Algorithms	CS	MATH	Mathematics	555-2000
MATH155	Calculus I	MATH			

Our full set of tables is now:

- **STUDENT(StudentID, StudentName)**
- **COURSE(CourseID, CourseName, DeptID)**
- **DEPARTMENT(DeptID, DeptName, DeptPhone)**
- **ENROLL_TEMP(StudentID, CourseID)**

Each FD has a key on the left-hand side, or the right-hand side is part of a key, so the schema is in 3NF.

Step 4: Check BCNF

We verify for each relation that every determinant is a superkey:

- In **STUDENT**, StudentID is the key and determinant.
- In **COURSE**, CourseID is the key and determinant.
- In **DEPARTMENT**, DeptID is the key and determinant.
- In **ENROLL_TEMP**, the key is (StudentID, CourseID) and there are no nontrivial FDs with a smaller determinant.

Therefore all relations are in BCNF. Our example has now been gradually converted from a non-1NF table to BCNF.

4NF Example: Multivalued Dependencies

4NF deals with independent multivalued attributes.

Initial 1NF Table

Consider the relation:

MOVIE(Title, Actor, Language)

Sample data:

Title	Actor	Language
M1	A1	English
M1	A1	French
M1	A2	English
M1	A2	French

For each movie:

- It has a set of actors.
- It has a set of languages.
- The choice of actor is independent of the choice of language.

We have multivalued dependencies:

$\text{Title} \twoheadrightarrow \text{Actor}$, $\text{Title} \twoheadrightarrow \text{Language}$.

The key of MOVIE is (Title, Actor, Language), so **Title** is *not* a superkey, and the relation violates 4NF.

4NF Decomposition

We decompose along the MVDs:

- **MOVIE_ACTOR**(Title, Actor)
- **MOVIE_LANG**(Title, Language)

Sample data:

MOVIE_ACTOR		MOVIE_LANG	
Title	Actor	Title	Language
M1	A1	M1	English
M1	A2	M1	French

Now the only MVDs have determinants that are keys in their respective relations, so both relations are in 4NF.

5NF Example: Join Dependencies

5NF addresses join dependencies not implied by keys.

Initial Relation

Consider:

SUPPLY(Supplier, Part, Project)

Interpretation: Supplier S can supply Part P to Project J .

Assume the following business rules:

- For each Supplier–Part pair (S, P) , we know whether the supplier can supply that part.
- For each Supplier–Project pair (S, J) , we know whether the supplier works on that project.
- For each Part–Project pair (P, J) , we know whether the part is used in that project.

Furthermore, we assume there is *no additional constraint* on specific triples (S, P, J) : whenever all three pairwise relationships hold, the triple is valid.

The key of SUPPLY is (Supplier, Part, Project).

There is a nontrivial join dependency:

$$\text{JD}^*(\{\text{Supplier, Part}\}, \{\text{Supplier, Project}\}, \{\text{Part, Project}\})$$

meaning SUPPLY can be reconstructed as the natural join of its three binary projections.

However, none of these projections contains a key of the original relation, so this violates 5NF.

5NF Decomposition

We decompose SUPPLY into the three projections:

- **SP(Supplier, Part)**
- **SJ(Supplier, Project)**
- **PJ(Part, Project)**

Sample data (illustrative):

SP		SJ		PJ	
Supplier	Part	Supplier	Project	Part	Project
S1	P1	S1	J1	P1	J1
S1	P2	S2	J1	P2	J1
S2	P2				

The original triples (Supplier, Part, Project) can be obtained as:

$$\text{SUPPLY} = SP \bowtie SJ \bowtie PJ.$$

There are no remaining nontrivial join dependencies that do not include a projection containing a key, so this decomposition is in 5NF.